# Define the first complete task

A release boundary that includes failure, recovery and deliberate omissions.

01 Choose a task with an observable finish.

02 Record the task and recovery paths.

03 Include the paths needed to trust the task.

04 List the rules the release depends on.

05 Make omissions explicit.

06 Check the boundary with the whole task.

# Choose a task with an observable finish.

Hypothetical booking release: a customer chooses a time and receives a confirmed appointment.

A selected slot without a confirmed booking is an incomplete task.

01 ENTRY

## Find an available time

Customer can understand which times are offered.

↓

02 COMMITMENT

## Choose and confirm

Required details and conditions are visible.

↓

03 COMPLETION

## Receive confirmation

The booking exists and the customer can find it again.

# Record the task and recovery paths.

Write the necessary paths before choosing the screens. Use the booking example to check what the task needs when something fails.

## Entry condition

What does the customer have or know when the task begins?

## Completion

What result can they inspect to confirm the task is finished?

## Recovery

What can they do when the expected path fails?

# Include the paths needed to trust the task.

A smaller release can still need an error path. Decide what the customer can do when the happy path fails.

| Moment | Must work | Can wait |
| --- | --- | --- |
| No time available | Explain availability and next option. | Advanced availability preferences. |
| Slot taken during booking | Keep entered details and offer another time. | A waitlist if not needed for the chosen task. |
| Confirmation delivery fails | Show the confirmed booking in the product. | Extra delivery channels. |

# List the rules the release depends on.

Unsettled rules can change the task more than another interface iteration.

### Rule or data source

What must be accurate for the task to work?

### Owner and readiness

Who can confirm it is usable for this release?

### Fallback

What will the customer see if it is unavailable?

# Make omissions explicit.

An omission needs a reason and a condition for reconsideration. Otherwise it can return as an argument in every review.

## Capability left out

What are we choosing not to include?

## Reason and risk

Why can the chosen task work without it? Who is affected?

## Review condition

What observation would make us add it?

# Check the boundary with the whole task.

Walk entry, confirmation and recovery together. Confirm the team knows which part proves the release is usable.

### Task and completion condition

Write what the person should be able to finish.

### Critical dependency

Which data, policy or service must be reliable?

### Release evidence

What will we inspect before calling this ready?